

## Complementação

### 1. Resumo da apresentação anterior:

**Introdução do problema e da engenharia de plataformas:** Vimos o problema inicial que introduziu a necessidade da engenharia de plataformas e os fundamentos utilizados nela.

**Expansão na metodologia:** Foi explicado em mais profundidade os conceitos como golden paths que criam guias para usuários, Self-service que permite desenvolvimento com menos dependências e DevEx, que oferece feedback e comunicação da experiência do desenvolvedor durante desenvolvimento e o uso da plataforma.

**Visão Prática:** Exploramos esses conceitos na prática por meio de um produto minimamente viável, em qual foi visto a interface que um usuário dessa plataforma iria utilizar, as informações relevantes da experiência do desenvolvedor, e o streamlining oferecido por Self-service no processo de criação.

**Limitações:** Discutimos as limitações e os pitfalls de engenharia de plataforma, como aumento de complexidade e carga cognitiva por meio de criar ainda mais abstração entre o usuário e o código, terminologia e documentação escassa, e a necessidade da aplicação não restrigente das ferramentas de engenharia de plataformas.

**Possível Futuro:** Teve Teorias do futuro da plataforma, como a utilização de AI para adicionar flexibilidade e adaptação, o paradigm shift da plataforma ser mais reconhecida como um produto, e o amadurecimento de frameworks e dados mais concretos.

**Conclusão:** Foi concluído que engenharia de plataformas aplicada efetivamente utilizar feedback e organização para adaptar a plataforma ao invés de adicionar mais ferramentas sem planejamento, tudo a fim de diminuir a carga cognitiva do desenvolvedor junto com automatizar e padronizar o desenvolvimento dentro da plataforma.

## 2. Artigo científico

### ***Platform Engineering and Internal Developer Portals: a Multivocal Literature Review – Anjum (2026)***

O artigo de Mateen Ali Anjum, publicado em maio de 2026 na *Frontiers in Computer Science*, realiza uma revisão multivocal sobre Engenharia de Plataformas e Internal Developer Portals. Diferente de uma revisão que utiliza apenas artigos científicos, o estudo também considera literatura produzida pela indústria, como relatórios técnicos e pesquisas de mercado. Isso acontece porque Platform Engineering ainda é uma área relativamente recente e grande parte das práticas utilizadas atualmente surgiu primeiro dentro das empresas e comunidades técnicas.

O estudo começou com 143 fontes candidatas e, após critérios de seleção e avaliação de qualidade, chegou a 88 fontes utilizadas na análise final. Destas, 44 foram classificadas como fontes acadêmicas de maior qualidade, 26 incluíram estudos acadêmicos de menor nível, teses e preprints, e 18 eram materiais da chamada literatura cinzenta. O autor separou as fontes dessa maneira justamente para não tratar uma pesquisa acadêmica e uma publicação de fornecedor como evidências de mesmo peso.

Esse ponto é importante para o debate porque o outro grupo criticou corretamente a forte influência da indústria sobre o tema. O próprio artigo confirma que existe uma situação diferente do que normalmente acontece em outras áreas da Engenharia de Software: em Platform Engineering, a prática avançou antes da pesquisa acadêmica. O estudo identifica aproximadamente três anos de diferença entre a consolidação do conceito dentro da indústria e os primeiros trabalhos acadêmicos focados diretamente em Engenharia de Plataformas.

Portanto, o artigo não tenta apresentar Engenharia de Plataformas como uma solução comprovada para todos os casos. Pelo contrário, ele ajuda a separar aquilo que já possui evidência mais consistente daquilo que ainda é principalmente uma prática promovida pela indústria.

### **Principais componentes identificados**

Ao comparar as fontes, Anjum identifica seis grupos de capacidades que aparecem repetidamente nas plataformas internas:

Developer Experience, catálogo de serviços, capacidades da plataforma, observabilidade, automação de workflows e governança/padronização.

Isso também ajuda a mostrar que uma IDP não deve ser entendida apenas como um portal ou uma interface gráfica. O portal é apenas uma maneira de disponibilizar capacidades. A Engenharia de Plataformas envolve um conjunto maior

de elementos que tentam reduzir o esforço necessário para que desenvolvedores encontrem informações, provisionam recursos, criem serviços ou utilizem infraestrutura.

O objetivo central continua sendo reduzir a complexidade enfrentada pelas equipes de produto sem necessariamente remover toda a complexidade existente no sistema.

### **Plataforma como produto**

Um dos pontos mais importantes do artigo é o conceito de Platform as a Product.

Nesse modelo, a plataforma é tratada como um produto interno e os desenvolvedores são tratados como seus usuários. A equipe de plataforma não deveria simplesmente decidir quais ferramentas as outras equipes devem utilizar.

Ela precisa compreender os fluxos de trabalho existentes, descobrir quais problemas realmente precisam ser resolvidos e acompanhar se as capacidades entregues estão sendo utilizadas.

O artigo relaciona isso ao princípio da “thinnest viable platform”, ou plataforma mínima viável: a plataforma deve fazer somente aquilo que realmente precisa fazer, evitando criar abstrações e funcionalidades desnecessárias. Esse ponto responde diretamente à crítica de over-engineering.

Uma plataforma que tenta resolver todos os problemas possíveis pode acabar se transformando em uma nova fonte de complexidade. O próprio Anjum identifica a abstração prematura como um padrão de falha: equipes constroem Golden Paths e automações sofisticadas antes mesmo de entender quais problemas dos desenvolvedores justificam esse investimento.

Nesse sentido, o artigo não contradiz a existência do risco apresentado pelo outro grupo. Ele explica como esse risco deve ser evitado.

### **Golden Paths e autonomia**

Outro ponto importante é a relação entre padronização e autonomia.

Os Golden Paths procuram fornecer um caminho pronto para tarefas que acontecem com frequência. Um novo micro serviço, por exemplo, pode receber automaticamente uma estrutura de projeto, pipeline, políticas básicas de segurança, observabilidade e configurações necessárias.

Porém, a literatura analisada por Anjum recomenda que esses caminhos funcionem como facilitadores e não como mandatos. O próprio estudo conclui que

plataformas devem preservar a autonomia do desenvolvedor e que Golden Paths rígidos podem gerar resistência.

Essa distinção é importante porque o outro grupo apresentou justamente o risco da chamada Golden Cage.

A crítica faz sentido quando uma organização transforma uma recomendação em obrigação. Porém, nesse caso, o problema não é a existência do Golden Path. O problema está na maneira como ele foi utilizado.

Um Golden Path bem aplicado procura fazer o caminho mais comum ser também o mais fácil, sem impedir situações que realmente precisam de uma solução diferente.

### **Como avaliar se a plataforma funciona**

Outro ponto muito relevante do artigo é que ele critica a ideia de avaliar Engenharia de Plataformas utilizando apenas uma métrica.

Anjum separa três grupos principais de indicadores:

DORA, relacionado ao desempenho de entrega, incluindo frequência de deploy, lead time, taxa de falha e tempo de recuperação, SPACE/DevEx, relacionado à experiência do desenvolvedor, satisfação, fluxo de trabalho, eficiência e comunicação e métricas específicas de plataforma, como adoção de self-service, tempo de onboarding, utilização dos Golden Paths e satisfação dos usuários internos.

Um problema identificado pelo artigo é que métricas específicas de Engenharia de Plataformas ainda possuem pouca validação acadêmica.

Isso significa que uma empresa não deveria concluir que sua plataforma funciona apenas porque aumentou a frequência de deploy, por exemplo. É possível aumentar uma métrica técnica enquanto a experiência dos desenvolvedores piora.

O artigo defende, portanto, uma combinação entre métricas técnicas e feedback dos próprios desenvolvedores.

### **Limitações e falta de evidências**

Talvez a parte mais importante para o debate seja que Anjum não esconde as limitações da área.

O estudo mostra que afirmações mais gerais, como a existência de relação entre qualidade das ferramentas e produtividade, possuem um conjunto razoável de evidências. Porém, quanto mais específica é a afirmação sobre uma funcionalidade de Platform Engineering, menor tende a ser a quantidade de estudos disponíveis.

Por exemplo, o artigo encontrou alguma evidência para melhorias na descoberta de serviços e onboarding, mas identifica práticas amplamente utilizadas comercialmente, como determinados scorecards de IDPs, sem evidência empírica revisada por pares demonstrando sua efetividade. Isso é importante porque permite uma posição mais equilibrada:

Engenharia de Plataformas não deve ser apresentada como uma solução cientificamente comprovada em qualquer contexto.

Porém, a ausência de validação para determinadas práticas também não comprova que a abordagem é ineficaz. Ela demonstra que existe uma área ainda em amadurecimento e que os resultados precisam ser avaliados dentro das próprias organizações.

### **Importância do artigo para nossa pesquisa**

O principal valor desse artigo para a complementação é justamente permitir responder às críticas sem precisar negá-las.

Ele reconhece praticamente todos os problemas importantes apresentados pelo outro grupo: over-engineering, risco de perda de autonomia, dificuldade de medir resultados, maturidade organizacional e influência da indústria.

Porém, o estudo apresenta esses problemas como condições que precisam ser consideradas durante a implantação, e não como provas de que Engenharia de Plataformas é uma abordagem inviável.

A principal interpretação que podemos retirar do artigo é que uma boa iniciativa de Platform Engineering depende mais de resolver problemas reais dos desenvolvedores do que da quantidade de funcionalidades existentes na plataforma.

### **3. Notícia recente**

Em um portal especializado em tecnologia, denominado InfoQ, que é voltado para profissionais da área de TI, publicou recentemente que um grupo está alcançando conformidade como um time de engenharia de plataforma, ajudando os desenvolvedores.

Essa equipe buscou implementar práticas de governança e conformidade, mas obteve resistência dos desenvolvedores devido aos novos processos impostos, por conta da complexidade dos fluxos de trabalho e documentações inadequadas.

Davide De Paolis, um profissional da tecnologia, relatou que após a criação da equipe de plataforma, foram planejadas iniciativas como um catálogo de serviços, uma plataforma interna para desenvolvedores e um centro de excelência em nuvem.

## Impactos relacionados

Entretanto, a adoção dessas mudanças gerou dificuldades como: desenvolvedores tendo que lidar com várias contas AWS, novas permissões, configurações adicionais e conceitos desconhecidos. Além disso, a documentação era extensa, desatualizada e difícil de consultar, prejudicando a experiência dos desenvolvedores e aumentando as solicitações de suporte.

Um exemplo foi a implementação de etiquetas (tags) nos recursos da AWS, utilizadas para identificar responsáveis, controlar custos e facilitar a governança. Uma primeira tentativa fracassou porque as regras eram vistas como burocracia. Na segunda tentativa, a equipe simplificou as exigências, definiu um conjunto mínimo de tags e utilizou ferramentas da AWS para prevenir e detectar recursos fora do padrão.

## Tendências relacionadas

A tendência é seguir uma estratégia de três etapas: prevenção: estabelecer regras e padrões para evitar problemas desde a criação dos recursos. Detecção: identificar recursos que não atendem às exigências. Comunicação: informar as equipes sobre os problemas e explicar a importância das mudanças antes de aplicar restrições mais rigorosas.

A governança deve priorizar os riscos mais importantes para o negócio, evitando sobrecarregar as equipes com muitas exigências simultâneas. Os desenvolvedores precisam compreender os motivos das mudanças e participar das decisões.

## Relação com os pontos negativos da engenharia de dados

O caso evidencia problemas que também podem ocorrer em ambientes de engenharia de dados e infraestrutura:

- Complexidade operacional: múltiplas contas, permissões e configurações aumentam o esforço de trabalho.
- Documentação inadequada: informações desatualizadas dificultam a utilização e a manutenção das plataformas.
- Burocracia na governança: regras excessivas ou impostas sem contexto podem prejudicar a produtividade.
- Custos e falta de rastreabilidade: a ausência de tags dificulta identificar responsáveis e atribuir custos aos recursos.
- Resistência à adoção: mudanças que não consideram as necessidades dos usuários podem gerar frustração e aumentar a demanda por suporte.

## 4. DEFESA TÉCNICA DA TECNOLOGIA

### Resistência Cultural e Atrito na Adoção

De fato, a apresentação que trouxe o argumento que fala sobre ser um problema organizacional com atrito está correto, com Da Paolis, o entrevistado da notícia recente feita pela infoQ sobre plataformas que ajudam desenvolvedores, descrevendo a mudança como “bagunçada no meio”, entretanto ele também adiciona que “apesar disso, vale a pena no fim.” para cuidar de problemas semelhantes ao Shadow IT, é possível utilizar a metodologia dele, em que ele diz que “transparência, empatia e implantação incremental são tão importantes quanto a tecnologia em si”, e que “se equipes verem conformidade como algo que ajuda eles a se moverem mais rápido e mais seguro, apoiados por detecção clara e guardrails sensatos, adoção segue naturalmente.”

Com isso, é possível concluir que uma comunicação clara que utiliza a experiência pessoal do desenvolvedor para considerar implementações de funcionalidades ausentes que iriam ser cobertas por Shadow IT, e trazer incentivos por meio de feedback e ferramentas relevantes e interessantes para os desenvolvedores.

### Golden Cages

Primeiramente eu gostaria de distinguir que a apresentação em si já traz um ponto de importância, que **Golden paths e golden cages são diferentes**. um golden path tem o risco de se tornar em um golden cage, mas não automaticamente se transforma em um, especialmente com implementação apropriada.

Já se foi feito o ponto durante o aprofundamento do artigo científico de Anjum (2026) sobre o tópico, que a diferença crucial entre os dois é a **Restrição do desenvolvedor e seu desenvolvimento**. Se a decisão final sempre for do desenvolvedor, com ferramentas para trabalhar em níveis de abstração menores ou sair do padrão quando necessário para o desenvolvimento, um golden cage não tem a capacidade de se manifestar.

A plataforma tendo restrições por meio de falta de funcionalidades durante o período transicional dela é algo discutido em mais profundidade durante o tópico de atrito na Adoção, mas é importante mencionar que de acordo com a conclusão feita por Anjum sobre um estudo de developer onboarding feito por Jasper e Green no google, atritos futuros como o developer onboarding tiveram grandes benefícios graças à implementação de golden paths, mostrando explicitamente que ao longo

termo que os golden paths reduzem atrito ao invés de ampliar ele.

## **5. Réplica do grupo apresentador**

A Engenharia de Plataformas pode, de fato, aumentar a complexidade quando é mal implementada. O contra-argumento apresentado pelo outro grupo destaca que uma Internal Developer Platform pode introduzir novas interfaces, configurações e abstrações para tarefas que antes eram mais simples, aumentando a carga cognitiva dos desenvolvedores.

A própria literatura reconhece esse risco, mas trata esse cenário como um problema de implementação, e não como uma característica inevitável da Engenharia de Plataformas.

No artigo de Anjum (2026), um dos pontos discutidos é o risco de criar abstrações e funcionalidades antes de entender quais problemas realmente precisam ser resolvidos. A ideia de uma plataforma interna não é concentrar o maior número possível de ferramentas e processos, mas oferecer apenas as capacidades que realmente reduzem atritos no trabalho dos desenvolvedores. Essa lógica se aproxima do conceito de uma plataforma mínima e evolutiva: primeiro identifica-se um problema real, depois cria-se uma solução para ele e, só então, a plataforma cresce conforme novas necessidades aparecem. Isso ajuda a evitar o chamado over-engineering e reduz a chance de a IDP se transformar em apenas mais uma camada de complexidade.

Esse ponto também aparece no trabalho de Ciancarini et al. (2025), que apresenta uma Internal Developer Platform auto-hospedada, de código aberto e estruturada com arquitetura baseada em microsserviços. A proposta mostra que uma plataforma pode ser construída de forma modular e extensível, permitindo que novas capacidades sejam adicionadas conforme necessário, em vez de concentrar tudo em uma estrutura rígida desde o início. Assim, a existência de uma camada adicional não significa automaticamente aumento de complexidade. O resultado depende principalmente do que essa camada substitui. Se ela transforma vários processos manuais, consultas de documentação, solicitações para outras equipes e configurações repetitivas em um fluxo único e automatizado, existe uma redução da complexidade percebida pelo desenvolvedor.

Outro ponto importante do contra-argumento é a falta de evidências empíricas sobre Engenharia de Plataformas. O próprio Anjum confirma que a área ainda possui uma produção acadêmica limitada quando comparada ao volume de conteúdo produzido pela indústria. O estudo analisou 88 fontes e identificou que apenas uma pequena parte delas vinha de venues acadêmicos de maior relevância tendo Platform Engineering como foco principal. Essa limitação é importante e não deve ser ignorada.

Porém, a falta de evidências suficientes não significa que existam evidências provando que a abordagem não funciona. O que existe atualmente é uma base científica ainda em amadurecimento, com níveis diferentes de evidência para cada prática. Algumas capacidades possuem resultados mais concretos, enquanto outras ainda são sustentadas principalmente por estudos de caso, experiências organizacionais e literatura da indústria.

O próprio trabalho original já reconhecia essa situação ao destacar a escassez de pesquisas empíricas, a influência da literatura industrial e a falta de métricas consolidadas para avaliar plataformas internas. Portanto, a conclusão mais adequada não é afirmar que Engenharia de Plataformas está totalmente comprovada, mas também não é afirmar que ela não funciona. O mais correto é entender que seus resultados dependem do contexto e precisam ser medidos dentro das próprias organizações.

Nesse sentido, a Developer Experience se torna um elemento importante para avaliar se uma plataforma realmente está cumprindo seu objetivo. Métricas técnicas, como tempo de entrega, frequência de deploy ou estabilidade, não deveriam ser utilizadas isoladamente. Elas precisam ser complementadas por informações sobre satisfação dos desenvolvedores, carga cognitiva, facilidade de uso, tempo de onboarding e utilização efetiva das funcionalidades da plataforma. O estudo de Greiler, Storey e Noda (2023), já utilizado no trabalho inicial, reforça essa ideia ao relacionar Developer Experience a produtividade, satisfação, engajamento e eficiência no trabalho.

Assim, os próprios pontos levantados pelo contra-argumento acabam reforçando uma visão mais cuidadosa da Engenharia de Plataformas. A abordagem não deve ser aplicada como uma solução universal nem adotada apenas porque está em destaque na indústria. Ela precisa partir de problemas concretos, crescer de forma gradual e ser avaliada continuamente com base nos resultados percebidos pelos desenvolvedores e pela organização.

Inclusive, a própria conclusão do grupo de oposição se aproxima bastante dessa interpretação. Eles afirmam que os riscos apresentados decorrem principalmente de implementações complexas, rígidas ou desalinhadas, e recomendam adoção baseada em problemas reais, Developer Experience e relação entre custo e benefício. Dessa forma, a pesquisa complementar permite mostrar que os problemas apresentados não necessariamente invalidam a Engenharia de Plataformas. Na verdade, eles representam justamente os principais cuidados que precisam ser considerados para que uma plataforma interna consiga atingir seu objetivo de reduzir complexidade, melhorar a experiência dos desenvolvedores e organizar melhor os processos de desenvolvimento.

## **Referências**

ANJUM, Mateen Ali. **Platform engineering and internal developer portals: a multivocal literature review**. *Frontiers in Computer Science*, v. 8, 2026. DOI: 10.3389/fcomp.2026.1814498.

CIANCARINI, Paolo; GIANCARLO, Raffaele; GRIMAUDO, Gennaro; MISSIROLI, Marcello; XIA, Tian Cheng. **The design and realization of a self-hosted and open-source agile internal development platform**. *IEEE Access*, v. 13, p. 79516–79533, 2025. DOI: 10.1109/ACCESS.2025.3564141.

GREILER, Michaela; STOREY, Margaret-Anne; NODA, Abi. **An actionable framework for understanding and improving developer experience**. *IEEE Transactions on Software Engineering*, v. 49, n. 4, p. 1411–1425, 2023. DOI: 10.1109/TSE.2022.3175660.

LINDERS, Ben. **Achieving compliance as a platform engineering team by helping developers**. *InfoQ*, 23 jul. 2026. Acesso em: 30 set. 2026.